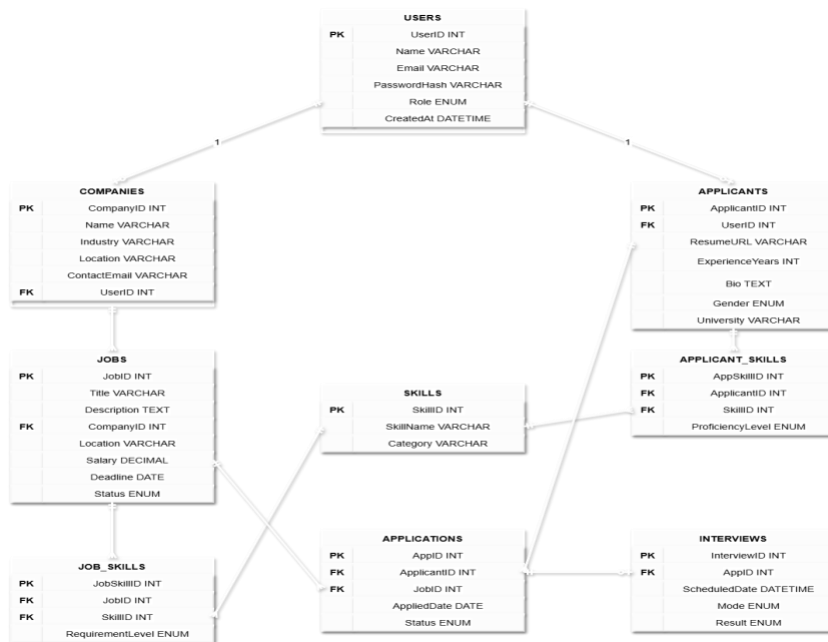


DATABASE DESIGN SPECIFICATION

Job Portal with Intelligent Filtering

Milestones	Date	Versions	Remarks
ERD & Schema Design	April 25, 2026	1.0	
Updated Schema, ERD and Workflow Diagram	May 12, 2026	2.0	

Entity Relation Diagram



1. Executive Summary

The Job Portal with Intelligent Filtering is a fully normalized MySQL relational database system engineered to digitize and streamline the complete recruitment lifecycle — spanning employer onboarding, job listing management, applicant registration, skill-based candidate matching, application submission, and interview scheduling.

The system's central design objective is to eliminate information silos and redundancy through Third Normal Form (3NF) normalization, while simultaneously enabling sophisticated, query-driven intelligence. Its Skill-Based Matching Engine compares job requirements against applicant proficiencies using a canonical Skills master table referenced by two dedicated junction tables — producing a transparent, percentage-based match score computable entirely through standard SQL.

Key design goals of this specification include:

- **Referential Integrity:** Foreign key constraints enforced across all 9 entities eliminate orphan records at the database level.
- **Role-Based Access Control:** A USERS table with an ENUM Role field distinguishes Admin, Employer, and JobSeeker participants, gating functionality accordingly.
- **Audit & Traceability:** Timestamped records (CreatedAt, AppliedDate, ScheduledDate) provide a complete audit trail across the recruitment pipeline.
- **Intelligent Filtering:** Normalized skill taxonomy enables SQL-driven candidate–job scoring without reliance on unstructured text search.
- **Pipeline Visibility:** ENUM-constrained status fields enforce valid state transitions across Applications and Interviews.

The schema is structured across three functional layers: Core Recruitment Management (identity, profiles, listings), the Skill-Based Matching Engine (taxonomy and junction tables), and the Application Lifecycle subsystem (transactional pipeline tracking).

2. Entity-Relationship Detail

The system comprises 9 interrelated entities organized into three functional layers. Each entity description below includes its primary key, attributes, data types, constraints, and inter-table relationships.

2.1 Core Recruitment Management

2.1.1 USERS — Central Authentication & Identity Store

The USERS table acts as the root authentication and identity store for every participant in the system. It manages role-based access control, ensuring that only appropriately privileged accounts can perform employer or administrative operations.

Field	Type	Constraint	Description
UserID	INT	PK, NOT NULL	Auto-incremented primary key; unique system-wide identifier for every account.
Name	VARCHAR	NOT NULL	Full display name of the user, shown on profiles and listings.
Email	VARCHAR	NOT NULL, UNIQUE	Login credential and contact address; UNIQUE prevents duplicate registrations.
PasswordHash	VARCHAR	NOT NULL	Bcrypt/Argon2 hash of credentials. Plaintext passwords are never persisted.
Role	ENUM	NOT NULL	Access tier: JobSeeker Employer Admin. Governs feature visibility throughout the application.
CreatedAt	DATETIME	NOT NULL	UTC timestamp of account creation; used for auditing and cohort analytics.

Relationships

- One-to-One (optional) with COMPANIES — an Employer-role user may register exactly one company profile.
- One-to-One (optional) with APPLICANTS — a JobSeeker-role user creates one applicant extended profile.

2.1.2 COMPANIES — Employer Organizational Profiles

Defines organizational entities that post job listings. Every COMPANIES record is owned by exactly one Employer-role user, preventing orphan listings and ensuring accountability. The separation of company data from the base USERS record follows 3NF normalization — employer-specific attributes are not imposed on all user types.

Field	Type	Constraint	Description
CompanyID	INT	PK, NOT NULL	Auto-incremented primary key; uniquely identifies each registered organization.
Name	VARCHAR	NOT NULL	Registered business name displayed on all associated job postings.
Industry	VARCHAR	NOT NULL	Sector classification (e.g., Technology, Finance) enabling industry-based filtering.
Location	VARCHAR	NOT NULL	Physical headquarters location supporting geographic job search filters.
ContactEmail	VARCHAR	NOT NULL	Official communication channel for applicant and administrative inquiries.

Field	Type	Constraint	Description
UserID	INT	FK → USERS, NOT NULL	References the Employer-role user who owns and administers this company profile.

Relationships

- Many-to-One with USERS — each company is owned by one employer user account.
- One-to-Many with JOBS — a single company can post multiple job listings.

2.1.3 APPLICANTS — Job Seeker Extended Profiles

Extends the base USERS record with job-seeker-specific profile data. This table separation adheres to normalization principles — only users carrying the JobSeeker role require resume, experience, and biographical attributes. Storing these on the USERS table would violate 2NF by introducing non-key dependencies for employer and admin rows.

Field	Type	Constraint	Description
ApplicantID	INT	PK, NOT NULL	Auto-incremented primary key uniquely identifying each applicant profile.
UserID	INT	FK → USERS, NOT NULL	Links the applicant profile to its parent USERS account (1:1 relationship).
ResumeURL	VARCHAR	NULL	File path or cloud URL to the uploaded resume document; optional at registration.
ExperienceYears	INT	NOT NULL	Total professional experience in years; used as a numeric filter in job search.
Bio	TEXT	NULL	Free-text professional summary displayed to employers on application review.
Gender	ENUM	NULL	Applicant gender identity (e.g., Male Female Other); used for diversity reporting and optional search filtering.
University	VARCHAR	NULL	Name of the applicant academic institution; supports education-based filtering and analytics.

Relationships

- Many-to-One with USERS — each applicant profile belongs to one user account.
- One-to-Many with APPLICANT_SKILLS — an applicant can possess and declare multiple skills.
- One-to-Many with APPLICATIONS — an applicant can submit multiple job applications.

2.1.4 JOBS — Primary Listing Entity

The JOBS table stores all details about open positions. Each job belongs to exactly one company and carries structured metadata enabling advanced search, salary-range queries, geographic filtering, and automatic expiry detection via the Deadline field.

Field	Type	Constraint	Description
JobID	INT	PK, NOT NULL	Auto-incremented primary key uniquely identifying each job listing.
Title	VARCHAR	NOT NULL	Position name displayed to applicants (e.g., Senior Backend Developer).
Description	TEXT	NOT NULL	Full-length listing detail including responsibilities, qualifications, and benefits.
CompanyID	INT	FK → COMPANIES, NOT NULL	Ensures every listing is associated with a valid, registered organization.
Location	VARCHAR	NOT NULL	Work location for the position; supports remote, hybrid, or on-site filtering.
Salary	DECIMAL	NULL	Offered compensation stored as decimal for precise salary-range query support.
Deadline	DATE	NOT NULL	Application closing date; WHERE Deadline >= CURDATE() filters active listings.
Status	ENUM	NOT NULL	Listing lifecycle state: Open Closed Draft. Restricts invalid state values.

Relationships

- Many-to-One with COMPANIES — each job belongs to exactly one company.
- One-to-Many with JOB_SKILLS — a job can require multiple skills at defined proficiency levels.
- One-to-Many with APPLICATIONS — a job receives multiple candidate applications.

2.2 Skill-Based Matching Engine

The intelligent filtering capability of the system is powered by a normalized skill-matching architecture. Rather than storing skills as unstructured text in job descriptions or applicant bios — which would be unsearchable and inconsistent — the schema maintains a centralized SKILLS master table referenced by two junction tables. This design enables the system to compute a transparent, percentage-based skill-match score through standard SQL JOIN and COUNT queries.

2.2.1 SKILLS — Canonical Skill Repository (Master Table)

A centralized repository of all professional skills recognized by the platform. This master list eliminates duplication and enforces consistent skill references across both job requirements and applicant profiles — ensuring that comparisons between the two are made against an identical taxonomy.

Field	Type	Constraint	Description
SkillIID	INT	PK, NOT NULL	Auto-incremented primary key uniquely identifying each skill definition.
SkillName	VARCHAR	NOT NULL, UNIQUE	Standardized skill name (e.g., Python, SQL, Agile). UNIQUE prevents duplicates.
Category	VARCHAR	NOT NULL	Logical grouping for UI filtering: Programming, Data Science, Soft Skills, etc.

Purpose

- Acts as the single source of truth for all skill definitions on the platform.
- Both JOB_SKILLS and APPLICANT_SKILLS reference this table via foreign key, ensuring skill comparisons are made against a consistent, controlled vocabulary.
- Prevents free-text skill entry that would produce mismatched comparisons (e.g., 'Python3' vs 'python' vs 'Python').

2.2.2 JOB_SKILLS — Job-to-Skill Junction Table

Resolves the many-to-many relationship between JOBS and SKILLS by recording which skills each job requires, along with the minimum expected proficiency level. This enables the matching engine to filter candidates not only by skill presence but also by competency grade.

Field	Type	Constraint	Description
JobSkillIID	INT	PK, NOT NULL	Auto-incremented surrogate primary key for each job-skill association record.
JobID	INT	FK → JOBS, NOT NULL	References the job listing that requires this skill.
SkillIID	INT	FK → SKILLS, NOT NULL	References the skill from the canonical SKILLS master table.
RequirementLevel	ENUM	NOT NULL	Minimum proficiency expected: Beginner Intermediate Expert.

Relationships

- Many-to-Many between JOBS and SKILLS — one job can require multiple skills; one skill can appear across many jobs.
- The RequirementLevel ENUM enables threshold-based filtering: exclude candidates who self-report Beginner proficiency for Expert-level requirements.

2.2.3 APPLICANT_SKILLS — Applicant-to-Skill Junction Table

Resolves the many-to-many relationship between APPLICANTS and SKILLS by recording which skills each applicant possesses and their self-assessed proficiency. This table is the second operand in the skill-match scoring query.

Field	Type	Constraint	Description
AppSkillID	INT	PK, NOT NULL	Auto-incremented surrogate primary key for each applicant-skill record.
ApplicantID	INT	FK → APPLICANTS, NOT NULL	References the applicant who claims this skill.
SkillID	INT	FK → SKILLS, NOT NULL	References the skill from the canonical SKILLS master table.
ProficiencyLevel	ENUM	NOT NULL	Self-declared competency: Beginner Intermediate Expert.

Relationships

- Many-to-Many between APPLICANTS and SKILLS — an applicant can possess many skills; one skill can be claimed by many applicants.
- ProficiencyLevel enables weighted scoring: Expert matches against Expert requirements score higher than Beginner matches.

2.3 Application Lifecycle & Status Tracking

The application subsystem manages the complete hiring pipeline from initial submission through final disposition. It provides real-time status visibility for applicants and structured candidate management for employers.

2.3.1 APPLICATIONS — Core Transactional Table

Each record in APPLICATIONS represents a single applicant's formal expression of interest in one specific job. The table tracks the application through a defined status pipeline with ENUM constraints that prevent invalid state values from being written to the database.

Field	Type	Constraint	Description
AppID	INT	PK, NOT NULL	Auto-incremented primary key uniquely identifying each application record.
ApplicantID	INT	FK → APPLICANTS, NOT NULL	The applicant who submitted this application.
JobID	INT	FK → JOBS, NOT NULL	The job listing to which this application was submitted.
AppliedDate	DATE	NOT NULL	Date of submission; enables recency-based sorting in employer dashboards.

Field	Type	Constraint	Description
Status	ENUM	NOT NULL	Current pipeline stage: Applied Shortlisted Rejected Hired.

Key Constraint — UNIQUE (ApplicantID, JobID)

- A composite UNIQUE constraint on (ApplicantID, JobID) prevents any applicant from submitting duplicate applications to the same job listing.
- This constraint is enforced at the database level, providing a hard guard independent of application-layer validation.
- Any INSERT that violates this constraint raises a database-level integrity error, ensuring clean, deduplicated application records.

2.3.2 INTERVIEWS — Application Lifecycle Extension

Extends the application lifecycle by recording scheduled interview events linked to specific applications. An interview record can only exist if a corresponding, valid application record exists — enforced by the foreign key on AppID. The One-to-One relationship with APPLICATIONS ensures a clean, unambiguous audit trail for the interview stage.

Field	Type	Constraint	Description
InterviewID	INT	PK, NOT NULL	Auto-incremented primary key uniquely identifying each interview record.
AppID	INT	FK → APPLICATIONS, NOT NULL	References the specific application for which this interview is scheduled.
ScheduledDate	DATETIME	NOT NULL	Planned date and time of the interview in UTC.
Mode	ENUM	NOT NULL	Interview format: Online Onsite. Restricts to these two valid values only.
Result	ENUM	NOT NULL	Outcome post-interview: Pending Passed Failed.

Relationships

- One-to-One with APPLICATIONS — each application can have at most one interview record.
- Mode ENUM (Online | Onsite) prevents free-text format values that would break reporting queries.
- Result ENUM (Pending | Passed | Failed) enables pipeline funnel analytics without ambiguous string matching.

3. Data Integrity & Business Logic

The schema enforces data quality and business rules at the database layer through a comprehensive constraint architecture. This eliminates reliance on application-layer validation alone and ensures data consistency even when the database is accessed directly via SQL tools or external integrations.

3.1 Foreign Key Constraints

All inter-entity references are enforced through foreign key declarations, preventing orphan records and maintaining referential consistency:

- Every APPLICATIONS record references a valid ApplicantID from APPLICANTS and a valid JobID from JOBS. An application cannot reference a deleted or non-existent entity.
- Every JOBS record references a valid CompanyID from COMPANIES; every COMPANIES record references a valid UserID from USERS.
- JOB_SKILLS and APPLICANT_SKILLS enforce valid references to their respective parent entities (JOBS/APPLICANTS) and to the SKILLS master table.
- Every INTERVIEWS record references a valid AppID from APPLICATIONS. Interviews cannot exist without a corresponding application — preventing scheduling without context.

3.2 UNIQUE Constraints

UNIQUE constraints enforce business rules that prevent logically invalid duplicate data:

UNIQUE (ApplicantID, JobID) — Applications Table

- Business Rule: An applicant may not submit more than one application to the same job listing.
- Implementation: A composite UNIQUE index on the (ApplicantID, JobID) column pair raises an integrity violation on any duplicate INSERT attempt.
- Impact: Guarantees that application count queries return accurate, deduplicated metrics. Employers see exactly one record per candidate per listing.
- Enforcement Level: Database constraint — independent of and supplementary to any application-layer duplicate-check logic.

UNIQUE (Email) — Users Table

- Business Rule: No two user accounts may share the same email address.
- Implementation: A single-column UNIQUE constraint on Users.Email raises an integrity error on duplicate registration attempts.
- Impact: Prevents account duplication, ensures email can safely serve as a login credential lookup key, and maintains one-to-one email–identity mapping.

3.3 NOT NULL Constraints

All primary keys and critical identification or transactional fields are declared NOT NULL, ensuring that no record can enter the database without its essential attributes:

- USERS: UserID, Name, Email, PasswordHash, Role, CreatedAt — all mandatory.
- JOBS: JobID, Title, Description, CompanyID, Location, Deadline, Status — all mandatory. Salary is nullable to support undisclosed compensation listings.

- APPLICANTS: UserID, ExperienceYears — mandatory. ResumeURL, Bio, Gender, and University are nullable to allow progressive profile completion.
- APPLICATIONS: AppID, ApplicantID, JobID, AppliedDate, Status — all mandatory.
- INTERVIEWS: InterviewID, AppID, ScheduledDate, Mode, Result — all mandatory.

3.4 ENUM Constraints — Valid State Enforcement

ENUM columns restrict fields to a predefined set of valid values, enforced at the storage engine level. This eliminates free-text status fields that would require application-layer string matching and produce inconsistent reporting data.

Table.Column	Valid ENUM Values	Business Meaning
USERS.Role	JobSeeker, Employer, Admin	Controls feature access gating; Admin has unrestricted platform access.
JOBS.Status	Open, Closed, Draft	Draft = unpublished; Open = accepting applications; Closed = no further applications.
APPLICATIONS.Status	Applied, Shortlisted, Rejected, Hired	Defines the four valid stages of the recruitment pipeline per application.
INTERVIEWS.Mode	Online, Onsite	Restricts interview format to two valid delivery modes; prevents free-text inconsistencies.
INTERVIEWS.Result	Pending, Passed, Failed	Pending = not yet conducted; Passed/Failed = post-interview outcome recorded.
JOB_SKILLS.RequirementLevel	Beginner, Intermediate, Expert	Minimum skill grade the employer expects for the position.
APPLICANT_SKILLS.ProficiencyLevel	Beginner, Intermediate, Expert	Applicant self-assessment; compared against RequirementLevel in match scoring queries.
APPLICANTS.Gender	Male, Female, Other	Optional demographic field; enables diversity analytics and applicant self-identification.

4. Reporting & Analytics Capabilities

The normalized schema is architected to support a full spectrum of operational and strategic reports through standard SQL aggregation, JOIN operations, and subqueries — without requiring denormalized reporting tables or external data warehousing.

4.1 Skill-Match Scoring

The schema's most distinctive analytical capability — computing a transparent percentage match between an applicant's declared skills and a job's required skills — is achievable entirely in SQL due to the normalized junction table architecture.

How Skill-Match Scoring Works

- Step 1 — Retrieve required skills: JOIN JOB_SKILLS with SKILLS on SkillID for a given JobID to obtain the full set of required skills and their RequirementLevel.
- Step 2 — Retrieve applicant skills: JOIN APPLICANT_SKILLS with SKILLS on SkillID for a given ApplicantID to obtain their declared skills and ProficiencyLevel.
- Step 3 — Compute intersection: COUNT the SkillIDs present in both result sets (matching skills) and divide by the total required skill COUNT for the job.
- Step 4 — Optional weighting: Apply proficiency-level weighting (Beginner=1, Intermediate=2, Expert=3) to score partial matches — a candidate with Expert Python scores higher than one with Beginner Python for an Expert-required role.
- Output: A percentage match score per (ApplicantID, JobID) pair, rankable and filterable to surface the most qualified candidates first.
- SQL Pattern: `SELECT COUNT(matched skills) / COUNT(required skills) * 100 AS MatchScore ... GROUP BY ApplicantID ORDER BY MatchScore DESC`

4.2 Hiring Funnel Analysis

The APPLICATIONS.Status ENUM field — combined with the timestamp columns AppliedDate and INTERVIEWS.ScheduledDate — enables full-funnel conversion analytics at the company, job, or platform level.

Supported Funnel Metrics

- Total Applications per Job: `COUNT(AppID) GROUP BY JobID` — identifies high-interest vs. low-visibility listings.
- Stage Conversion Rates: `COUNT WHERE Status = 'Shortlisted' / COUNT WHERE Status = 'Applied' per JobID` — measures recruiter screening efficiency.
- Interview-to-Hire Rate: `COUNT WHERE Interview.Result = 'Passed' / COUNT(Interviews)` — measures interview quality and offer conversion.
- Time-to-Hire: `DATEDIFF(INTERVIEWS.ScheduledDate, APPLICATIONS.AppliedDate)` — average days from application to interview stage per company.
- Rejection Analysis: `COUNT WHERE Status = 'Rejected' at each pipeline stage` — identifies where candidate drop-off is highest.
- Employer Dashboard: Active listings, pending applications, scheduled interviews per CompanyID — aggregated from JOBS, APPLICATIONS, and INTERVIEWS in a single JOIN query.

4.3 Additional Supported Reports

- Skill Demand Trends: Frequency of each SkillID across all Open JOBS — identifies the most in-demand competencies on the platform.
- Top Applicants by Match Score: Ranked list of applicants per job sorted by skill-match percentage — the core output of the intelligent filtering engine.

- Applicant Activity Dashboard: Applications submitted, statuses, and interview results per ApplicantID — real-time job seeker pipeline view.
- Industry-Level Analytics: Applications and hires grouped by COMPANIES.Industry — supports platform-level labour market reporting.
- Expired Listing Cleanup: WHERE Jobs.Deadline < CURDATE() AND Status = 'Open' — automated identification of listings requiring status update to 'Closed'.

5. Relational Schema Summary

The following table provides a consolidated reference of all 9 entities in the Job Portal schema, including their primary keys, foreign key dependencies, functional layer classification, and key inter-table relationships.

Entity	PK	Foreign Keys	Layer	Key Relationship
Users	UserID	—	Core	Parent of Companies & Applicants
Companies	CompanyID	UserID	Core	1:N with Jobs
Applicants	ApplicantID	UserID	Core	1:N with Applicant_Skills, Applications
Jobs	JobID	CompanyID	Core	1:N with Job_Skills, Applications
Skills	SkillID	—	Matching	Referenced by Job_Skills, Applicant_Skills
Job_Skills	JobSkillID	JobID, SkillID	Matching	M:N between Jobs and Skills
Applicant_Skills	AppSkillID	ApplicantID, SkillID	Matching	M:N between Applicants and Skills
Applications	AppID	ApplicantID, JobID	Lifecycle	1:1 (opt.) with Interviews
Interviews	InterviewID	AppID	Lifecycle	1:1 with Applications

All entities are normalized to Third Normal Form (3NF). Foreign key constraints enforce referential integrity across all inter-entity relationships. ENUM constraints restrict status and proficiency fields to their defined valid value sets. UNIQUE constraints on USERS.Email and the (ApplicantID, JobID) pair in APPLICATIONS enforce critical business rules at the storage layer.

6. Summary of Design Principles

Principle	Implementation in This Schema
3NF Normalization	All non-key attributes depend only on the primary key. Skill data centralized in SKILLS master table; role-specific attributes in COMPANIES/APPLICANTS.
Referential Integrity	Foreign key constraints across all 9 entities prevent orphan records at the storage engine level.
Business Rule Enforcement	ENUM constraints on 7 columns and UNIQUE constraints on 2 column pairs enforce domain rules without application-layer dependency.
Intelligent Matching	Normalized junction tables (JOB_SKILLS, APPLICANT_SKILLS) enable SQL-computable skill-match percentage scoring.
Pipeline Traceability	ENUM Status fields on APPLICATIONS and INTERVIEWS enforce valid state transitions; timestamp columns support time-based analytics.
Security by Design	PasswordHash stores only hashed credentials; Role ENUM gates feature access; UNIQUE(Email) prevents account duplication.

7. System Workflow Diagrams

This section presents the end-to-end operational workflows for both primary user roles in the Job Portal system. Each diagram maps the sequential steps a user performs to the specific database entities touched at each stage, providing a visual trace of how the relational schema supports the complete recruitment lifecycle.

7.1 Job Seeker Workflow

The job seeker workflow spans nine stages from account registration through final hire confirmation. The flow begins with USERS table insertion on registration, extends to APPLICANTS and APPLICANT_SKILLS during profile and skill setup, queries JOBS, COMPANIES, JOB_SKILLS, and APPLICANT_SKILLS for skill-match scoring, creates an APPLICATIONS record on submission, and concludes with an INTERVIEWS record and a final APPLICATIONS status update (Hired or Rejected).

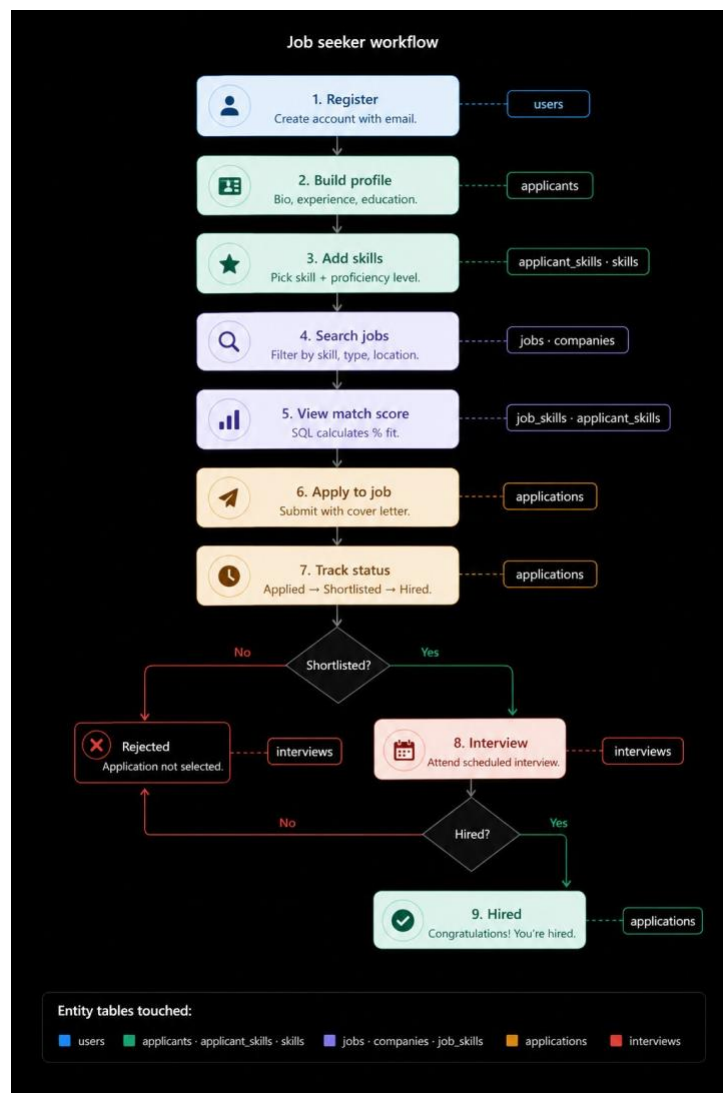


Figure 7.1 — Job Seeker Workflow: 9-stage pipeline from registration to hire, mapped to database entities.

7.2 Employer / Recruitment Platform Workflow

The employer workflow covers nine stages from employer account creation through final candidate disposition. Following USERS and COMPANIES record creation, the employer inserts a JOBS record and attaches required skills via JOB_SKILLS. Incoming APPLICATIONS are ranked using a skill-match percentage computed by joining APPLICANT_SKILLS against JOB_SKILLS. Shortlisted candidates progress to an INTERVIEWS record, after which the employer marks the APPLICATIONS.Status as either Hired or Rejected.

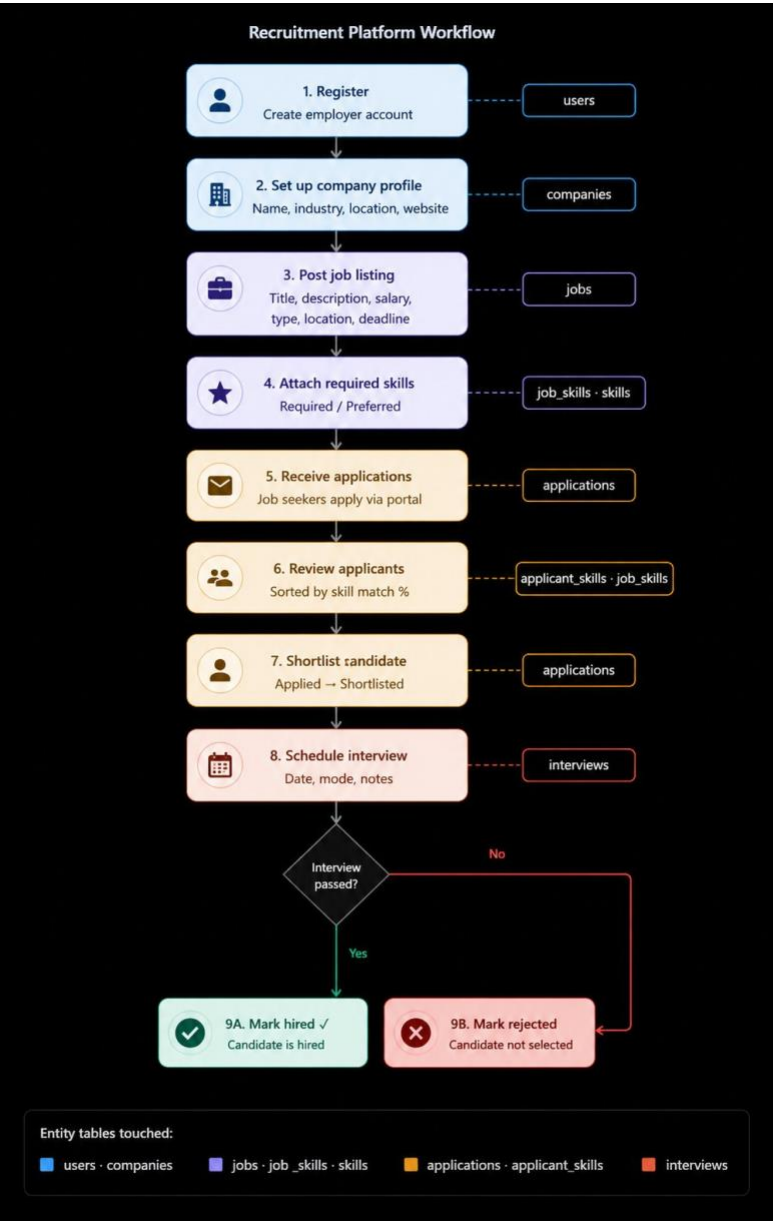


Figure 7.2 — Employer / Recruitment Platform Workflow: 9-stage pipeline from company setup to hire/reject disposition.